# Unit 6A: Symbolic Computing, Polymorphism, and the Type System

## Table of contents


**1 Project Preliminaries** **1**

**2 Symbolic Versus Numeric Computation** **2**

**3 Polynomial Arithmetic** **3**
- 3.1 Elementary Algebra . . . 3
- 3.2 Elementary Number Theory . . . 5
  - 3.2.1 Greatest Common Divisor . . . 8
  - 3.2.2 Inversion in $\mathbb{Z}_m$ . . . 10
- 3.3 The Polynomial Ring $\mathcal{R}[x]$ . . . 10
  - 3.3.1 Addition in $\mathcal{R}[x]$ . . . 11
  - 3.3.2 Multiplication in $\mathcal{R}[x]$ . . . 11

**4 Implementing Arithmetic in $\mathcal{R}[x]$** **12**
- 4.1 Dense Versus Sparse Representation . . . 12
- 4.2 Term Representation . . . 13
- 4.3 Abstract Polynomial . . . 15
  - 4.3.1 Addition of Abstract Polynomials . . . 15
  - 4.3.2 Multiplication of Abstract Polynomials . . . 16
- 4.4 Dense Polynomial . . . 17
  - 4.4.1 Addition of Dense Polynomials . . . 18
  - 4.4.2 Multiplication of Dense Polynomials . . . 19
- 4.5 Sparse Polynomial . . . 19
  - 4.5.1 Addition of Sparse Polynomials . . . 19
  - 4.5.2 Multiplication of Sparse Polynomials . . . 20
- 4.6 Repeated Squaring . . . 20
- 4.7 Division in $\mathbb{Z}_p[x]$ . . . 21

**5 Heaps** **25**
- 5.1 Underlying Representation . . . 26
- 5.2 Implemeting a Heap in Julia . . . 28
  - 5.2.1 Base Interfaces . . . 29
  - 5.2.2 Pop / Sink . . . 29
  - 5.2.3 Push / Swim . . . 31
- 5.3 Heapfiy . . . 33


# 1 Project Preliminaries

Clone the repository using the instructions from Project B assignment specification.

Assuming `project_to_project_dir` is the path to `MATH2504_2026_ProjectB` then the code base is initialized by doing.

```
Pkg.develop(path = path_to_project_dir)
using PolynomialAlgebra
```

Now we have access to basic polynomials and algebraic operations acting on polynomials that are used throughout these notes.

```
x = x_poly(PolynomialDense{Int64})
z = zero(PolynomialDense{Int64})
p = (3x^2-x)*x
```

```
3 x³ - x²
```

## 2 Symbolic Versus Numeric Computation

Numerical computation is fast, well understood, and typically sufficient. NASA requires only *15 digits* of $\pi$ for interplanetary exploration and *39 digits* is adequate to estimate the volume of the known universe accurate to within a hydrogen nucleus.

Nonetheless, to write $\sqrt{2}$ as a decimal requires infinitely many digits whereas computer memory is finite. Thereby any numerical/float/machine representation of $\sqrt{2}$ is necessarily off by some error $\varepsilon > 0$.

**Example 2.1.**

$$\begin{aligned} \sqrt{2}_{10} &= 1.414213562 & (\sqrt{2}_{10})^2 &= 1.999999999 \\ \sqrt{2}_{15} &= 1.41421356237310 & (\sqrt{2}_{15})^2 &= 2.00000000000001 \end{aligned}$$

This error is not necessarily a problem in isolation. The problem is that a computation usually involves a sequence of arithmetic operations, each of which can propagate or amplify the error.

For example, *adding* the inexact numbers $x$ and $y$ *doubles* the inexactness

$$(x \pm \varepsilon) - (y \pm \varepsilon) = (x - y) \pm 2\varepsilon.$$

This is catastrophic for common algorithms like Gaussian elimination because when we require $x - y = 0$ we actually get $x - y = 2\varepsilon$. **Numerical instability**, left unchecked, will produce nonsense answers (like a lower triangular matrix with no zeros).

In *symbolic computation* there is no numerical instability because all values are represented *exactly*. Any computer algebra system will accommodate arbitrarily large integers, fractions (e.g., $\frac{1}{3}$ not $0.333\ldots$), and *arithmetic on polynomials*. This final inclusion is significant because having polynomial arithmetic enables us to extend the number system from rational numbers to *algebraic numbers*.

An *algebraic number* is any number that is a root of a non-zero polynomial with rational coefficients. In this setting $x^2 - 2$ is an encoding for $\pm\sqrt{2}$. Given an algebraic number $\alpha$, there is a *unique* (up to scalar) polynomial of minimum degree (called the *minimal polynomial*) that has $\alpha$ as a root.

**Example 2.2.** The *minimal polynomial* for

$$\alpha = \sqrt{1 + \sqrt[3]{7 + \sqrt[5]{9}}}$$

is

$$\begin{aligned} &x^{30} - 15\,x^{28} + 105\,x^{26} - 490\,x^{24} + 1785\,x^{22} - 5313\,x^{20} + 13195\,x^{18} \\ &-28170\,x^{16} + 51795\,x^{14} - 81935\,x^{12} + 113043\,x^{10} - 131880\,x^8 \\ &+129920\,x^6 - 107520\,x^4 + 61440\,x^2 - 32777. \end{aligned}$$

Because polynomials can be evaluated at algebraic numbers via remainders,

$$\begin{aligned}\operatorname{rem}(f = x^3 - 2x + 2,\, x - 3) &= 23 = f(3),\\ \operatorname{rem}(f = x^2,\, x^2 - 2) &= 2 = f(\pm\sqrt{2}),\\ \operatorname{rem}(f = x^2,\, x^2 + 1) &= -1 = f(i).\end{aligned}$$

arithmetic on polynomials can be used to do arithmetic in field extensions of $\mathbb{Q}$ containing algebraic numbers.

# 3 Polynomial Arithmetic

The purpose of this module and project is to build a library for performing *exact* arithmetic in rings with efficient algorithms. In particular we are interested in the *ring of integers*

$$\mathbb{Z} = \{0, -1, 1, 2, -2, 3, -3, ...\}$$

and *polynomials*

$$\mathcal{R}[x] = \left\{ \sum_{k=0}^{N} a_k x^k \colon k \in \mathbb{N},\, a_k \in \mathcal{R} \right\}.$$

In order to work with polynomials it is first necessary to establish terms and definitions.

## 3.1 Elementary Algebra

Let $\mathbb{N} = \{0, 1, 2, ...\}$ be the set of *natural numbers*, $\mathbb{Z} = \{..., -2, -1, 0, 1, 2, ...\}$ be the *integers* and $\mathbb{P}$ be the set of odd primes.

**Definition 3.1** (Ring)**.** A ring $\mathcal{R}$ is a set with addition $(+)$ and multiplication $(\cdot)$ satisfying the following conditions called the *ring axioms*:

- $(\mathcal{R}, +)$ is an abelian group, that is:
  - $(+)$ is associative *and* commutative,
  - $\exists b \in \mathcal{R}$ called the *additive identity* satisfying $\forall a \in \mathcal{R}\ a + b = a$,
  - $\forall a \in \mathcal{R}\ \exists b \in \mathcal{R}$ called the *additive inverse* satisfying $a + b = 0$.
- $(\mathcal{R}, \cdot)$ is a monoid, that is:
  - $(\cdot)$ is associative, and
  - $\exists b \in \mathcal{R}$ called the *multiplicative identity* satisfying $\forall a \in \mathcal{R};\ a \cdot b = b \cdot a = a$
- Multiplication *distributes* over addition. That is, $\forall a, b, c \in \mathcal{R}$:
  - $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$
  - $(b + c) \cdot a = (b \cdot a) + (c \cdot a)$

**Example 3.1.**

1. The $\mathcal{R} = \{0\}$ (the *zero ring*) is a trivial ring.
2. The naturals $\mathbb{N}$ do *not* form a ring as they lack additive identity.
3. Arithmetic on the clock forms a ring.

**Definition 3.2** (Zero Divisor)**.** An element $a \in \mathcal{R}$ is called a *zero divisor* if there is nonzero $b \in \mathcal{R}$ such that $ab = 0$.

**Definition 3.3** (Integral Domain)**.** An *integral domain* is a nonzero commutative ring without zero divisors.

**Example 3.2.**

- The integers $\mathbb{Z}$ form an integral domain.
- 'Clock arithmetic' does not form an integral domain because, for instance, $3 \cdot 4$ is 12 which is the zero.

**Definition 3.4** (Field)**.** A ring $\mathcal{R}$ is also a *field* when each non-zero element has a *multiplicative inverse*. Equivalently, $\mathcal{R}$ is a field when

$$\forall a \in \mathcal{R}^{\neq 0};\ \exists b \in \mathcal{R} :\ a \cdot b = 1.$$

**Example 3.3.** For instance, the *rationals*

$$\mathbb{Q} = \left\{ \frac{a}{b} \,:\, a, b \in \mathbb{Z},\, b \neq 0 \right\}$$

is a field because $\frac{a}{b} \cdot \frac{b}{a} = 1$ when $a, b \neq 0$.

**Theorem 3.1** (Division Algorithm)**.** *Let* $a, b \in \mathbb{Z}^{\geq 0}$ *with* $b > 0$*. There are* unique $q$ *and* $r$ *(called the* quotient *and* remainder*) satisfying*

$$a = b \cdot q + r$$

*with* $0 \leq r < b$*.*

To prove Theorem 3.1 we must establish $a$ and $b$ *exist* and are *uique*.

Note: Theorem 3.1 does not actually require $a \geq 0$. We do so here to simplify the presentation.

*Existence.* Existence of $a$ and $b$ is demonstrated *constructively*. That is, we give an algorithm that produces them.

```
function remainder(a::T, b::T) where T <: Integer
    a < b && return a
    return remainder(a-b, b)
end
```

```
remainder(15, 5), remainder(23, 7)
```

```
(0, 2)
```

```
function quo(a::T, b::T) where T <: Integer
    a < b && return 0
    return 1 + quo(a-b, b)
end
```

```
quo(15, 5), quo(23, 7)
```

```
(3, 3)
```

What remains to be shown is that the algorithm for `remainder` and `quo` *terminates* and is *correct*. This can be done with induction but we omit this part here. □

*Uniqueness.* Suppose there are two ways to write $a$ as

$$a = b \cdot q + r \qquad \text{and} \qquad a = b \cdot q' + r'$$

with $0 \leq r, r' < b$.

We have

$$\begin{aligned} b \cdot q + r &= b \cdot q' + r' \\ &\implies b \cdot (q - q') = r' - r \\ &\implies b \mid (r - r') \end{aligned}$$

which means $(r - r')$ is a multiple of $b$ such that $0 \leq r - r' < |b|$. This is only possible when $r - r' = 0$ or $r = r'$ and it follows $q = q'$ which contradicts our assumption. □

**Example 3.4.** Of course, in Julia we have `%` and `÷` for remainder and and quotient.

```
for (a,b) in [(15, 5), (4, 101), (125,4 )]
    q, r = a ÷ b, a % b
    println("$a = $q $b + $r")
end
```

```
15 = 3 5 + 0
4 = 0 101 + 4
125 = 31 4 + 1
```

Note that in Python `//` is the quotient whereas in Julia `//` defines a rational type.

```
23 // 7, float(23//7)  #The first // is a rational type #!!! Note in Python // is ÷
```

```
(23//7, 3.2857142857142856)
```

Incidentally, another way to express the quotient/remainder relation is by way of "proper" fractions.

$$23 = 3 \cdot 7 + 2 \iff \frac{23}{7} = 3 + \frac{2}{7}$$

```
(23 ÷ 7), (23 % 7) // 7
```

```
(3, 2//7)
```

```
23 // 7 == (23 ÷ 7) + (23 % 7) // 7
```

```
true
```

## 3.2 Elementary Number Theory

Working with exact fractions eliminates numerical instability, but introduces another problem called *expression swell*. The numerators and denominators of fractions can become very large during intermediate calculations, making exact computation expensive.

One way to mitigate this is to perform computations in smaller rings called *residue classes*, and then use techniques such as the Chinese Remainder Theorem and *p*-adic lifting to recover a solution to the original problem.

For example, rather than computing directly with the fraction $\frac{1}{2}$, we can work modulo a prime:

$$\frac{1}{2} \equiv 3 \bmod 5 \qquad \text{becase} \qquad 1 \equiv 2 \cdot 3 \bmod 5$$

Thus, arithmetic involving potentially large rational numbers can temporarily be replaced by arithmetic with small integers, after which *rational reconstruction* can be used to recover the original fractions.

It is therefore useful to develop some elementary number theory for working in $\mathbb{Z}_p$, since some of our polynomial algorithms require the coefficient ring to be a field and we want to avoid the expression swell associated with working over $\mathbb{Q}$.

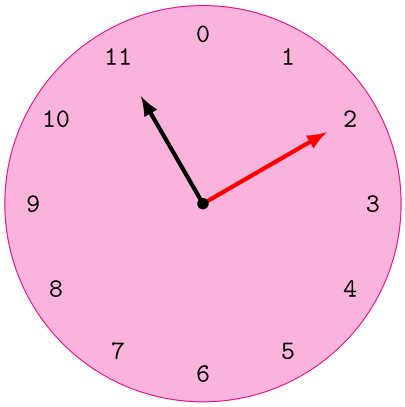
**Definition 3.5** (Residue Classes)**.** The sets $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ for $n \in \mathbb{N}$ are rings called *residue classes* when addition and multiplication are done 'mod n'. This is sometimes called 'clock arithmetic' as three hours past eleven is two because $11 + 3 \equiv 2 \bmod 12$.

We say/write

$$a \equiv b \bmod c$$

when $a$ *is congruent to* $b$ *modulo* $c$. Also:

$$a \equiv b \bmod c \iff \text{rem}(a, c) = b \iff \text{a \% c = b}$$




$$11 + 3 \equiv 2 \mod 12$$

Figure 1: Clock arithmetic

**Example 3.5.** We have already seen something similar with overflow. In particular, working in an 8-bit register is essentially doing arithmetic modulo $2^8$.

```
UInt8(2^8-3) + UInt8(4)
```

```
0x01
```

```
(2^8-3 + 4) % 2^8
```

```
1
```

**Definition 3.6** (Symmetric Mod)**.** We may also use negatives values, here $\mathbb{Z}_n = \{-\text{quo}(n,2),\ 0,\ \ldots,\ \text{quo}(n,2)\}$. For instance, $\mathbb{Z}_7 = \{-3, -2, -1, 0, 1, 2, 3\}$ and

$$5 \equiv -2 \text{ smod } 7.$$

We do not use this notation and simply move use `mod` as `smod` when we want as there is no effect on the theory and presentation.

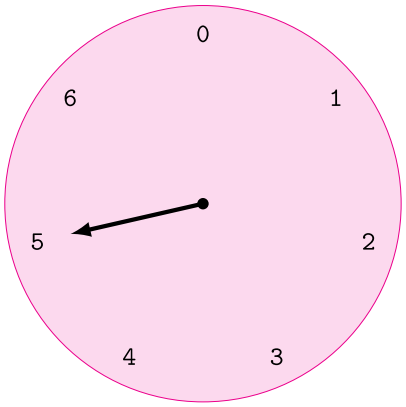
**Example 3.6.** $\mathbb{Z}_6 = \{0, \ldots, 5\}$ has the following *addition table*

```
n = 6
Z_n = 0:(n-1)
typeof(Z_n)
A = [(x+y) % n for y in Z_n, x in Z_n]
```

```
6×6 Matrix{Int64}:
 0  1  2  3  4  5
 1  2  3  4  5  0
 2  3  4  5  0  1
 3  4  5  0  1  2
 4  5  0  1  2  3
 5  0  1  2  3  4
```

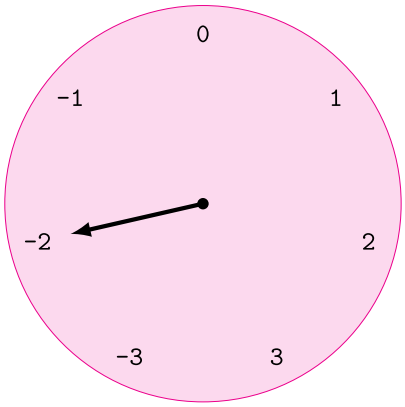
Notice:

1. the addition table is symmetric (equal to its transpose) which can only happen when the addition is commutative,
2. the additive identity is 0,




$-2 \equiv 5 \mod 7$

(a) Regular modulo




$5 \equiv -2 \operatorname{smod} 7$

(b) Symmetric modulo

Figure 2: Comparison of modulo and symmetric modulo arithmetic.

3. each column (and row) has 0 and thereby each element has an additive inverse: for instance, the third column indicates that 2 has additive inverse 4 and correspondingly $2 + 4 \equiv 0 \mod 6$.

```
(A .== 0)
```

```
6×6 BitMatrix:
 1  0  0  0  0  0
 0  0  0  0  0  1
 0  0  0  0  1  0
 0  0  0  1  0  0
 0  0  1  0  0  0
 0  1  0  0  0  0
```

```
additive_inverses = [findfirst((A .== 0)[:,k+1])-1 for k in Z_n]
println(additive_inverses)  # print horizontally instead of vertically
```

```
[0, 5, 4, 3, 2, 1]
```

**Example 3.7** (Multiplication Table)**.** $\mathbb{Z}_6$ has the following *multiplication table*

```
M = [(x*y) % n for y in Z_n, x in Z_n]
```

```
6×6 Matrix{Int64}:
 0  0  0  0  0  0
 0  1  2  3  4  5
 0  2  4  0  2  4
 0  3  0  3  0  3
 0  4  2  0  4  2
 0  5  4  3  2  1
```

Notice:

1. the multiplicative identity is 1,
2. *not* all rows contain one, which means there are some elements that do *not* have *multiplicative inverse*. For instance there is no $b \in \mathbb{Z}_6$ such that $2 \cdot b \equiv 1 \mod 6$.

```
println([(2*y) % n for y in Z_n])
```

```
[0, 2, 4, 0, 2, 4]
```

```
mult_inverses = Z_n[[sum((M .== 1)[:,k+1]) > 0 for k in Z_n]]
println("Elements with a multiplicative inverse: $mult_inverses")
println("Elements without a multiplicative inverse: $(setdiff(Z_n, mult_inverses))")
```

```
Elements with a multiplicative inverse: [1, 5]
Elements without a multiplicative inverse: [0, 2, 3, 4]
```

Notice $\mathbb{Z}_6$ is *not* a field because, recall, there is no multiplicative inverse for 2. However, $\mathbb{Z}_p$ is a field when $p$ is a prime number.

**Example 3.8.** $\mathbb{Z}_5 = \{0, \ldots, 4\}$ is a field and has the following multiplication table.

```
n = 5
Z_n = 0:(n-1)
M = [(x*y) % n for y in Z_n, x in Z_n]
```

```
5×5 Matrix{Int64}:
 0  0  0  0  0
 0  1  2  3  4
 0  2  4  1  3
 0  3  1  4  2
 0  4  3  2  1
```

```
mult_inverses = Z_n[[sum((M .== 1)[:,k+1]) > 0 for k in Z_n]]
println("Elements with a multiplicative inverse: $mult_inverses")
println("Elements withoout a multiplicative inverse: $(setdiff(Z_n,mult_inverses))")
```

```
Elements with a multiplicative inverse: [1, 2, 3, 4]
Elements withoout a multiplicative inverse: [0]
```

Notice every column (and row) contains 1 asides the first column (and row) which corresponds to 0. For instance, the third column indicates the inverse of 3 is 2 and correspondingly $3 \cdot 2 \equiv 1 \bmod 5$.

### 3.2.1 Greatest Common Divisor

The *greatest common divisor* is one of the fundamental operations of a computer algebra system. For instance, we need `gcd` to reduce fractions to their canonical form.

**Definition 3.7** (GCD)**.** Let $a, b \in \mathbb{Z}$ not both 0. We say $g \in \mathbb{Z}$ is the **greatest common divsor** of $a$ and $b$, denoted $\gcd(a, b)$, when

1. $g \mid a$ and $g \mid b$ ($g$ is a common divisor),
2. $h \mid a \ \wedge \ h \mid b \implies h \mid g$ (greatest),
3. $g > 0$ (required for uniqueness)

**Example 3.9** (GCD)**.**

- The $\gcd(6, 4) = \gcd(2 \cdot 3, 2 \cdot 2) = 2$.
- The $\gcd(6, 0) = \gcd(1 \cdot 6, 0 \cdot 6) = 6$.

The is an in-built `gcd()` function for integers (and a few more types) but we will soon make our own.

```
using Random; Random.seed!(10)

a_test = *(rand(1:100,5)...)
b_test = *(rand(1:100,5)...)
@show a_test, b_test
```

```
Base.gcd(a_test, b_test)
```

```
(a_test, b_test) = (4145382, 1056517560)
```

```
18
```

#### 3.2.1.1 Euclid's Algorithm

The **Euclidean Algorithm** computes the gcd of two integers. (Actually the Euclidean Algorithm computers gcds for any two elements of a *Euclidean Domain*). The algorithm exploits the following property of the gcd:

**Lemma 3.1.** *Let* $a, b \in \mathbb{Z}^{>0}$ *and* $a = b \cdot q + r$ *with* $r \in [0, b)$. *Then*

1. $\gcd(b, a) = \gcd(a, b)$,
2. $\gcd(a, b) = \gcd(r, b)$,
3. $\gcd(a, b) = \gcd(a - b, b)$.

It is straightforward to turn this lemma into an algorithm:

```
function euclid_alg(a::T, b::T) where T <: Integer
    (b == 0) && return a
    return euclid_alg(b, a % b)  # Rule 1 and 2
end
```

```
euclid_alg(2, 2*2), euclid_alg(3*7, 2*3), euclid_alg(2*2, 13)
```

```
Base.gcd(a_test, b_test), euclid_alg(a_test,b_test)
```

```
(18, 18)
```

#### 3.2.1.2 Extended Euclid's Algorithm

The **Extended Euclidean Algorithm** in addition to the $\gcd(a, b)$, also computes $s$ and $t$ (called the Bézout coefficients) such that

$$as + bt = \gcd(a, b).$$

```
function i_ext_euclid_alg(a:: T,b:: T) where T <: Integer
    (a == 0) && return b, 0, 1
    g, s, t = i_ext_euclid_alg(b % a, a)
    return g, t - (b ÷ a)*s, s
end
```

```
pretty_print_egcd((a,b),(g,s,t)) = println("$a×$s + $b×$t = $g = gcd($a, $b)") #\times + [TAB]

for (a,b) in [(4,12), (9,12), (4,13)]
    pretty_print_egcd((a,b), i_ext_euclid_alg(a,b))
end
```

```
4×1 + 12×0 = 4 = gcd(4, 12)
9×-1 + 12×1 = 3 = gcd(9, 12)
4×-3 + 13×1 = 1 = gcd(4, 13)
```

Note there is an in-built gcdx.

```
Base.gcdx(a_test,b_test)
```

```
(18, 2477299, -9720)
```

### 3.2.2 Inversion in $\mathbb{Z}_m$

Notice the Extended Euclidean Algorithm computes inverses in $\mathbb{Z}_m$. Given $a \in \mathbb{Z}_m^{\neq 0}$ the egcd$(a, m)$ returns $s$ and $t$ such that

$$a \cdot s + m \cdot t = gcd(a, m).$$

Taking the entire equation mod$m$ we get

$$\begin{aligned} \gcd(a, m) &\equiv a \cdot s + m \cdot t \mod m \\ \implies \gcd(a, m) &\equiv a \cdot s + 0 \mod m \\ \implies \gcd(a, m) &\equiv a \cdot s \mod m. \end{aligned}$$

Provided $\gcd(a, m) = 1$ (i.e. "coprime" or "relatively prime") we get

$$a \cdot s \equiv 1 \bmod m \implies a^{-1} \equiv s \bmod m.$$

**Example 3.10.** To find the inverse of $5 \in \mathbb{Z}_{13}$ do:

```
a, m = 5, 13
i_ext_euclid_alg(5, 13)
```

```
(1, -5, 2)
```

```
inverse_mod(a,m) = mod(i_ext_euclid_alg(a,m)[2], m)
```

```
inverse_mod (generic function with 1 method)
```

```
i = inverse_mod(a,m)
```

```
8
```

```
mod(a*i, m)
```

```
1
```

## 3.3 The Polynomial Ring $\mathcal{R}[x]$

By this point in your mathematics career you have surely worked with *polynomials*. A polynomial is something like $x^2 + 2x + 1$ but *not* like $\frac{x^2+1}{x-1}$ or $x \sin(x)$.

**Definition 3.8.** Let $\mathcal{R}$ be a ring and $a \in \mathcal{R}[x]$. Let

$$a = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

with $a_n \neq 0$.

- $a$ is called a **polynomial** in $x$ and $a_k x^k$ are its **terms**.
- The **degree** of $a$ is $n$:
$$\deg(a) = n.$$
- The **leading coefficient** of $a$ is $a_n$:
$$\mathrm{lc}(a) = a_n.$$
- The **leading term** of $a$ is $a_n x^n$:
$$\mathrm{lt}(a) = a_n x^n.$$

By convention we let $\mathrm{lc}(0) = \mathrm{lt}(0) = 0$ and $\deg(0) = 0$.

### 3.3.1 Addition in $\mathcal{R}[x]$

Polynomial addition is defined by *combining like terms*. If

$$f = \sum_{i=0}^{n} a_i x^i \qquad \text{and} \qquad g = \sum_{i=0}^{m} b_i x^i,$$

then

$$f + g = \sum_{i=0}^{\max(n,m)} (a_i + b_i) x^i,$$

where any coefficient not present in one of the polynomials is treated as zero.

In other words, polynomial addition combines terms of the same degree by adding their coefficients.

**Example 3.11** (Addition of Polynomials)**.** Let

$$f = 3x^3 + 2x + 1 \qquad \text{and} \qquad g = 2x^2 + 4x + 5.$$

Aligning terms of the same degree gives

$$\begin{array}{rcccccccc} f & = & 3x^3 & + & 0x^2 & + & 2x & + & 1 \\ g & = & 0x^3 & + & 2x^2 & + & 4x & + & 5 \\ \hline f+g & = & 3x^3 & + & 2x^2 & + & 6x & + & 6 \end{array}$$

Here the "missing" $x^2$ term in $f$ and the missing $x^3$ term in $g$ are treated as having coefficient zero.

**Proposition 3.1.** *When $\mathcal{R}$ is a ring then $\mathcal{R}[x]$ is a group.*

*Sketch.* Follows from the defintion of addition over $\mathcal{R}[x]$.

By our construction, when $f, g \in \mathcal{R}[x]$ then $f + g \in \mathcal{R}[x]$.

When $a_i, b_i, c_i \in \mathcal{R}$ then commutativity follows from

$$a_i + b_i = b_i + a_i$$

and associativity from

$$(a_i + b_i) + c_i = a_i + (b_i + c_i).$$

$0 \in R[x]$ is the additive identity.

The additive inverse of $f = \sum_{i=0}^{n} a_i x^i$ is $g = \sum_{i=0}^{n} (-a_i) x^i$ where $-a_i$ is the additive inverse of $a_i$. □

### 3.3.2 Multiplication in $\mathcal{R}[x]$

Multiplication is defined by multiplying every term of one polynomial by every term of the other and then *combining like terms*. If

$$f = \sum_{i=0}^{n} a_i x^i \qquad \text{and} \qquad g = \sum_{j=0}^{m} b_j x^j,$$

then

$$fg = \sum_{k=0}^{n+m} \left( \sum_{i+j=k} a_i b_j \right) x^k.$$

**Example 3.12** (Multiplication of Polynomials)**.** Let

$$f = 2x^2 + 3x + 1 \qquad \text{and} \qquad g = x + 4.$$

then by definition

$$\begin{aligned} fg &= (2 \cdot 1)x^3 + (2 \cdot 4 + 3 \cdot 1)x^2 + (3 \cdot 4 + 1 \cdot 1)x^1 + (1 \cdot 4)x^0 \\ &= 2x^3 + 8x^2 + 3x^2 + 12x + x + 4. \end{aligned}$$

*Sketch.* By our construction when $f, g \in \mathcal{R}[x]$ then $fg \in \mathcal{R}[x]$.

Associativity and multiplicative distribution follows from the associativity of the coefficient ring.

The muliplicative identify is $1 \in \mathcal{R}[x]$. □

The following properties are consequences of our definition.

**Proposition 3.2** (Polynomial Multiplication)**.** *When $\mathcal{R}$ is an* integral domain *(a nonzero commutative ring in which the product of any two nonzero elements is nonzero) and $a, b \in \mathcal{R}[x]^{\neq 0}$ then*

1. $\deg(ab) = \deg(a) + \deg(b)$,
2. $\operatorname{lc}(ab) = a_b \cdot b_m = \operatorname{lc}(a) \cdot \operatorname{lc}(b)$,
3. $\operatorname{lm}(ab) = \operatorname{lm}(a) \cdot \operatorname{lm}(b)$,
4. $\operatorname{lt}(ab) = \operatorname{lt}(a) \cdot \operatorname{lt}(b)$.

*Sketch.*

$$\begin{aligned} a \times b &= (a_n x^n + \cdots + a_0)(b_m x^m + \cdots + b_0) \\ &= a_n \cdot b_m x^{n+m} + \cdots + a_0 \cdot b_0 \end{aligned}$$

□

**Proposition 3.3.** *When $\mathcal{R}$ is a ring then $\mathcal{R}[x]$ is a ring.*

# 4 Implementing Arithmetic in $\mathcal{R}[x]$

We now move to the practical considerations of representing polynomials in Julia so that we may operate on them.

## 4.1 Dense Versus Sparse Representation

There are two standard representations for polynomials in computer algebra that have their own benefits and drawbacks.

The first is a *dense* representation which is simple but requires us to explicitly store zero terms. In a dense representation the terms of a polynomial are stored in a *vector* where the term at index $i$ is the one of degree $i - 1$.

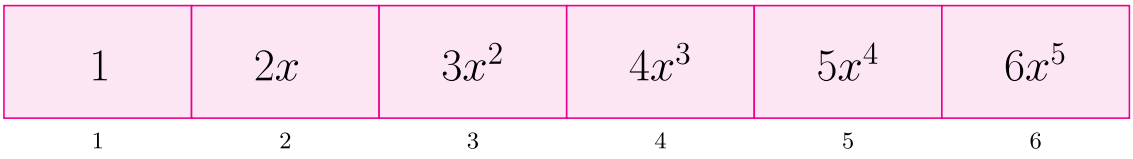



Figure 3: Dense representation of $6x^5 + 5x^4 + 4x^3 + 3x^2 + 2x + 1$

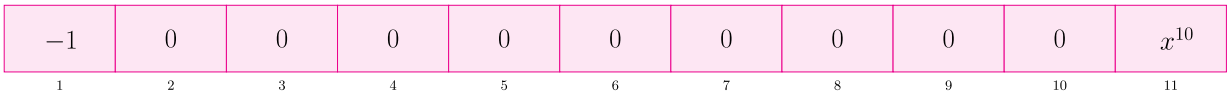



Figure 4: Dense representation of $x^{10} - 1$

The second is a *sparse* representation which does *not* store zero terms. The trade-off is that locating a term of a particular degree is more difficult. In a sparse representation, the nonzero terms of a polynomial are stored in a max-heap ordered by term degree.

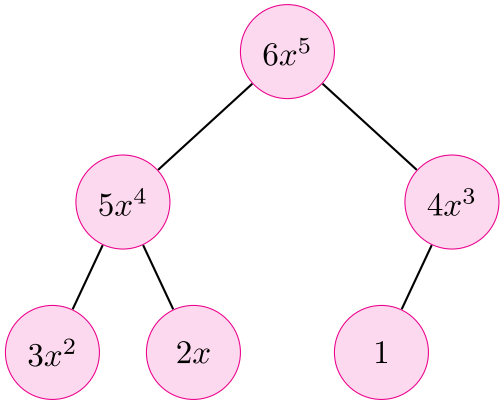



Figure 5: Sparse representation of $6x^5 + 5x^4 + 4x^3 + 3x^2 + 2x + 1$

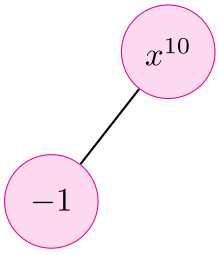



Figure 6: Sparse representation of $x^{10} - 1$

The important point is that we want to support both representations and be able to choose between them depending on our needs. At the same time, we do not want to repeatedly implement identical functionality when it is unnecessary. We would also like both representations to provide the same interface: for example, polynomial addition and operations for finding the leading term or degree.

This suggests that we should separate the idea of a polynomial from the particular data structure used to represent it by defining an abstract type `Polynomial`, with dense and sparse polynomial types as its subtypes.

## 4.2 Term Representation

Both representations require us to store *terms*, so we begin by defining their representation. We represent a Term as a struct with a coefficient and a degree. An inner constructor enforces the *representation invariant* that every term must satisfy.

```
struct Term{C <: Number} # structs are immutable by default
    coeff::C
    degree::Int

    function Term{C}(coeff::C, degree::Int) where C
        degree < 0 && error("Degree must be non-negative")
        new(coeff, degree)
    end
end
```

```
t = Term{Int64}(3, 2)
typeof(t)
```

```
Term{Int64}
```

```
t = Term{BigInt}(BigInt(3), 2)
typeof(t)
```

```
Term{BigInt}
```

```
t = Term{Int64}(3, -1)
```

```
Degree must be non-negative
```

#### 4.2.0.1 Outer Constructors

For *inferring* the coefficient/degree types when creating terms.

```
function Term(coeff::C, degree::Int) where C
    Term{C}(coeff, degree)
end
```

Now we can instead do.

```
t = Term(3, 2)
typeof(t)
```

```
Term{Int64}
```

```
t = Term(BigInt(3), 2)
typeof(t)
```

```
Term{BigInt}
```

For constructing constants.

```
function Term(coeff::C) where C
    Term(coeff, 0)
end
```

```
Term(3) == Term(3, 0)
```

```
true
```

For `zero` and `one` which are common functions in `Base` that make sense to define for the Term type as well.

```
function Base.zero(::Type{Term{C}})::Term{C} where C
    Term(zero(C), 0)
end
Base.zero(t::Term) = zero(typeof(t))
```

```
function Base.one(::Type{Term{C}})::Term{C} where C
    Term(one(C), 0)
end
Base.one(t::Term) = one(typeof(t))
```

## 4.3 Abstract Polynomial

Our "top-level" polynomial type is given by the following. Notice that we allow the coefficient type to be any subtype of `Number`, since we want to represent polynomials over a general coefficient ring $\mathcal{R}$, rather than restricting ourselves to $\mathbb{Z}[x]$ or $\mathbb{Q}[x]$.

```
abstract type Polynomial{C <: Number} end
```

There are also many outter constructors defined but the following will be of particular utility.

```
function x_poly(::Type{P})::P where {C, P <: Polynomial{C}}
    return P([Term(one(C),1)])
end
x_poly(p::P) where {P <: Polynomial} = x_poly(P)
```

It allows us to capitalize on Julia's implied multiplcation of numbers and symbols to instantiate polynomials in a natural way without having to build a parser.

```
x = x_poly(PolynomialDense{Int64})
@show f = x^2 + 1
dump(f)
```

```
f = x ^ 2 + 1 = x² + 1
PolynomialDense{Int64}
  terms: Array{Term{Int64}}((3,))
    1: Term{Int64}
      coeff: Int64 1
      degree: Int64 0
    2: Term{Int64}
      coeff: Int64 0
      degree: Int64 1
    3: Term{Int64}
      coeff: Int64 1
      degree: Int64 2
```

### 4.3.1 Addition of Abstract Polynomials

At the abstract Polynomial level, addition can be generically described as repeatedly adding a single term to a polynomial:

$$(((f + g_0) + g_1) + \cdots + g_m).$$

This reduces the problem of adding two polynomials to the simpler problem of repeatedly adding a single term.

However, the implementation of $f+g_0$ depends on the underlying representation of $f$. Thus, while polynomial addition $f + g$ can be defined at the abstract level, adding a single term to a polynomial cannot. That operation must instead be implemented separately for each concrete polynomial representation.

**Example 4.1** (Addition of Polynomials)**.** Let

$$f = 3x^2 + 2x + 1 \qquad \text{and} \qquad g = 4x^3 + x^2 + 5.$$

Then

$$\begin{aligned} f + g &= ((f + 4x^3) + x^2) + 5 \\ &= ((4x^3 + 3x^2 + 2x + 1) + x^2) + 5 \\ &= (4x^3 + 4x^2 + 2x + 1) + 5 \\ &= 4x^3 + 4x^2 + 2x + 6 \end{aligned}$$

```
"""
Add a polynomial and a term.

This must be overridden by concrete subtypes.
"""
function Base.:(+)(p::Polynomial{C}, t::Term{C}) where C
    not_implemented_error(p, "Polynomial + Term")
end
Base.:(+)(t::Term, p::Polynomial) = p + t
```

```
"""
Add two polynomials of the same concrete subtype.
"""
function Base.:(+)(p1::P, p2::P)::P where {P <: Polynomial}
    p = deepcopy(p1)
    for t in p2
        p += t
    end
    return p
end
```

```
"""
Add a polynomial and an integer.
"""
Base.:(+)(p::Polynomial{C}, n::Integer) where C = p + Term(C(n),0)
Base.:(+)(n::Integer, p::Polynomial) = p + n
```

```
"""
The negative of a polynomial.
"""
Base.:(-)(p::P) where {P <: Polynomial} = P(map((pt)->-pt, p))
```

```
"""
Subtraction of two polynomials (of the same concrete subtype).
"""
function Base.:(-)(p1::P, p2::P)::P where {P <: Polynomial}
    return p1 + (-p2)
end
```

### 4.3.2 Multiplication of Abstract Polynomials

At the abstract `Polynomial` level, multiplication can be described as multiplying one polynomial by each term of the other and then adding the results:

$$f(g_0 + \cdots + g_m) = fg_0 + \cdots + fg_m.$$

This reduces the problem of multiplying two polynomials to the simpler problem of multiplying a polynomial by a single term and then adding the resulting polynomials.

**Example 4.2** (Multiplication of Polynomials)**.** Let

$$f = 2x^2 + 3x + 1 \qquad \text{and} \qquad g = x + 4.$$

Then

$$fg = f \cdot x + f \cdot 4,$$

so

$$\begin{aligned} fg &= (2x^2 + 3x + 1)x + (2x^2 + 3x + 1)4 \\ &= (2x^3 + 3x^2 + x) + (8x^2 + 12x + 4) \\ &= 2x^3 + 11x^2 + 13x + 4. \end{aligned}$$

Thus, once we define how to compute

```
function Base.:(*)(t::Term, p::P)::P where {P <: Polynomial}
```

polynomial multiplication can be implemented by summing the products of `f` with each term of `g`.

```
"""
Multiply two polynomials (of the same concrete subtype).
"""
function Base.:(*)(p1::P, p2::P)::P where {P <: Polynomial}
    p_out = P()
    for t in p1
        new_summand = (t * p2)
        p_out = p_out + new_summand
    end
    return p_out
end
```

```
"""
Multiplication of polynomial and term.
"""
function Base.:(*)(t::Term, p::P)::P where {P <: Polynomial}
    return iszero(t) ? P() : P(map((pt)->t*pt, p))
end
Base.:(*)(p::Polynomial, t::Term)::Polynomial = t*p
```

```
"""
Multiplication of polynomial and an integer.
"""
function Base.:(*)(p::P, n::Integer)::P where {C, P <: Polynomial{C}}
    Term(C(n),0) * p
end
Base.:(*)(n::Integer, p::Polynomial) = p*n
```

```
(2x^2+3x+1)*(x+4)
```

```
2 x³ + 11 x² + 13 x + 4
```

## 4.4 Dense Polynomial

```
struct PolynomialDense{C <: Number} <: Polynomial{C}
    terms::Vector{Term{C}}

    # Inner constructor of 0 polynomial
    PolynomialDense{C}() where C = new{C}([zero(Term{C})])
```

```
    # Inner constructor of polynomial based on arbitrary list of terms
    function PolynomialDense{C}(vt::Vector{Term{C}}) where C

        # Filter the vector so that there is not more than a single zero term
        vt = filter((t)->!iszero(t), vt)
        if isempty(vt)
            vt = [zero(Term{C})]
        end

        #First set all terms with zeros
        max_degree = maximum((t)->t.degree, vt)
        terms = [zero(Term{C}) for i in 0:max_degree]

        # now update based on the input terms
        for t in vt
            terms[t.degree + 1] = t # +1 accounts for 1-indexing
        end
        return new{C}(terms)
    end
```

### 4.4.1 Addition of Dense Polynomials

Addition of dense polynomials can use the generic definition already given for the abstract Polynomial type, provided that we define how to add a Term to a dense polynomial.

```
function Base.:(+)(p::PolynomialDense{C}, t::Term{C}) where C
    p = deepcopy(p)
    if t.degree > degree(p)
        push!(p, t)
    else
        if !iszero(p.terms[t.degree + 1]) #+1 is due to indexing
            p.terms[t.degree + 1] += t
        else
            p.terms[t.degree + 1] = t
        end
    end
    trim!(p)  # remove trailing zero coefficients
    return p
end
```

This is sufficient to obtain a working implementation of polynomial addition.

```
x = x_poly(PolynomialDense{Int64})
(3x^2 + 2x + 1) + (2x^5 + 5x^2 - 2x + 4)
```

```
2 x  + 8 x² + 5
```

However, one of the main benefits of the dense representation is that adding two dense polynomials reduces essentially to vector addition. We need only account for vectors of different lengths and remove any trailing zero terms from the result.

**Example 4.3** (Addition of Dense Polynomials)**.** Consider

$$f = 1 + 2x + 3x^2 \qquad \text{and} \qquad g = 4 - 2x - 3x^2.$$

Their dense representations are

$$\langle 1, 2, 3 \rangle \qquad \text{and} \qquad \langle 4, -2, -3 \rangle.$$

Adding componentwise gives

$$\langle 1, 2, 3\rangle + \langle 4, -2, -3\rangle = \langle 5, 0, 0\rangle.$$

The trailing zero coefficients are then removed, giving

$$\langle 5\rangle$$

which is the dense representation of the constant polynomial 5.

**Project Task**

Implement addition on the `PolynomialDense` type that exploits the underlying vector representation.

```
function Base.:(+)(p1::P, p2::P)::P where {C, P <: PolynomialDense{C}}
```

#### 4.4.2 Multiplication of Dense Polynomials

No specialized implementation is required because the abstract version suffices.

### 4.5 Sparse Polynomial

Define a subtype of `Polynomial` to represent sparse polynomials.

**Project Task**

```
struct PolynomialSparse{C} <: Polynomial{C}
    # ...
end
```

#### 4.5.1 Addition of Sparse Polynomials

**Project Task**

Implement addition of a `Term` to a sparse polynomial so that the polynomial addition defined for the abstract `Polynomial` type also works for sparse polynomials.

```
function Base.:(+)(p::P, t::Term{C}) where {C, P <: PolynomialSparse{C}}
```

**Project Task**

Implement addition of sparse polynomials directly using a merging stategy.

```
function Base.:(+)(p1::P, p2::P)::P where {C, P <: PolynomialSparse{C}}
```

The following is pseudocode for this task that should be adapted to use `push` and `pop` operations instead. Notice, for instance, that `pop!(f)` removes `f`'s leading term.

```
def ADDITION:
    Input   a, b sparse polynomials in ZZ[x]
    Output  a + b

    c = 0

    while a != 0 or b != 0

        if deg(a) == deg(b)
            c += lt(a) + lt(b) # addtion on *terms* ax^n + bx^n = (a+b)x^n
            a, b = a-lt(a), b-lt(b)

        if deg(a) > deg(b)
            c += lt(a)
            a = a-lt(a)

        if deg(a) < deg(b)
            c += lt(b)
            b = b-lt(b)

    return c
```

### 4.5.2 Multiplication of Sparse Polynomials

**Project Task**

Implement multlication on the `PolynomialSparse`.

```
function Base.:(*)(t::Term, p::P)::P where {P <: PolynomialSparse}
```

If you wish, you may utilize a strategy for multiplying sparse polynomials that exploits sparsity given by Johnson in 1974. Otherwise, just do repeated adding.

## 4.6 Repeated Squaring

Suppose we want to compute

$$w^{103}$$

First notice that

$$103 = 64 + 32 + 4 + 2 + 1 = (1100111)_2,$$

where $(x)_2$ denotes the binary representation of $x$. Therefore,

$$w^{103} = w^{64} w^{32} w^4 w^2 w.$$

We can obtain the required powers by repeated squaring:

$$\begin{aligned} w^2 &= w \cdot w \\ w^4 &= w^2 \cdot w^2 \\ w^8 &= w^4 \cdot w^4 \\ w^{16} &= w^8 \cdot w^8 \\ w^{32} &= w^{16} \cdot w^{16} \\ w^{64} &= w^{32} \cdot w^{32} \end{aligned}$$

Thus, rather than computing $w^{103}$ using roughly 103 multiplications, we require only 6 squarings to obtain the powers of two, followed by 4 multiplications to combine the five powers that appear in the binary expansion.

For example, to compute $8^{17^3}$, we have

$$17^3 = 4913 = (1001100110001)_2$$

and hence

$$17^3 = 2^{12} + 2^9 + 2^8 + 2^5 + 2^4 + 2^0.$$

We therefore perform 12 squarings to obtain the powers

$$8^{2^0}, 8^{2^1}, \ldots, 8^{2^{12}},$$

reducing modulo 17 after every squaring. We then multiply together the required powers:

$$8^{2^{12}} \cdot 8^{2^9} \cdot 8^{2^8} \cdot 8^{2^5} \cdot 8^{2^4} \cdot 8^{2^0}.$$

For comparison, the naive method requires thousands of multiplications, while repeated squaring requires only a number proportional to the number of binary digits in the exponent.

**Project Task**

Implement `^` for the abstract `Polynomial` type.

```
function Base.:(^)(p::P, n::Integer)::P where {P <: Polynomial}
```

The following is pseudocode for this task.

```
def POW:
    Input   x in RR, m in ZZ such that m >= 0.
    Ouput   x^m

    let m = b0*2^0 + b1*2^1 + ... + bk*2^k with bi in {0, 1}
    # i.e. determine the binary representation of m

    ans, w = 1, x

    for i from 0 to k do
        if bi == 1 then
            ans *= w
        w *= w

    return ans
```

## 4.7 Division in $\mathbb{Z}_p[x]$

We are unable to *divide* in $\mathcal{R}[x]$. Consider the following operation in $\mathbb{Z}[x]$

$$\frac{x^2}{2x} = \frac{1}{2}x.$$

Even though $x^2$ and $2x$ are from $\mathbb{Z}[x]$ that their division is in $\mathbb{Q}[x]$. This is because $\mathbb{Z}$ is not a *field* but $\mathbb{Q}$ is.

It turns out $\mathbb{F}[x]$ is a **Euclidean Domain** when $\mathbb{F}$ is a field. Re-consider the division, but this time let us do the calcuation in $\mathbb{Z}_7[x]$

$$\begin{aligned}\frac{x^2}{2x} &\equiv \frac{1}{2}x \bmod 7 \\ &\equiv 2^{-1}x \bmod 7 \\ &\equiv 4x \bmod 7.\end{aligned}$$

Note that $\frac{1}{2} \equiv 4 \bmod 7$.

**Theorem 4.1** (Polynomial Division)**.** *Let* $a, b \in \mathbb{F}[x]$*,* $b \neq 0$*. There exists* unique *polynomials* $q, r \in \mathbb{F}[x]$ *such that*

$$a = bq + r \qquad \textit{and} \qquad r = 0 \qquad \textit{or} \qquad \deg r < \deg b$$

*Uniqueness.* Let $a = bq_1 + r_1 = bq_2 + r_2$ with $\deg r_1, \deg r_2 < \deg b$.

Consider

$$\begin{aligned} bq_1 + r_1 = bq_2 + r_2 &\implies b(q_1 - q_2) = (r_2 - r_1) \\ &\implies b|(r_2 - r_1) \qquad\qquad \text{But } \deg r_1, r_2 < \deg b \\ &\implies r_2 - r_1 = 0 \\ &\implies r_2 = r_1 \end{aligned}$$

Therefore $b(q_1 - q_2) = 0$. But since $\mathbb{F}[x]$ is an integral domain it cannot have zero divisors. As we assumed $b \neq 0$ this implies

$$(q_1 - q_2) = 0 \implies q_1 = q_2. \square$$

$\square$

The division algorithm is basically the same as *synthetic division* that students of calculus are taught.

Consider dividing

$10x^2 + 4x + 1$

by

$5x - 3$

in $\mathbb{Z}_{11}[x]$.

- $5x - 3 \equiv 5x + 8 \mod 11$

Long division:

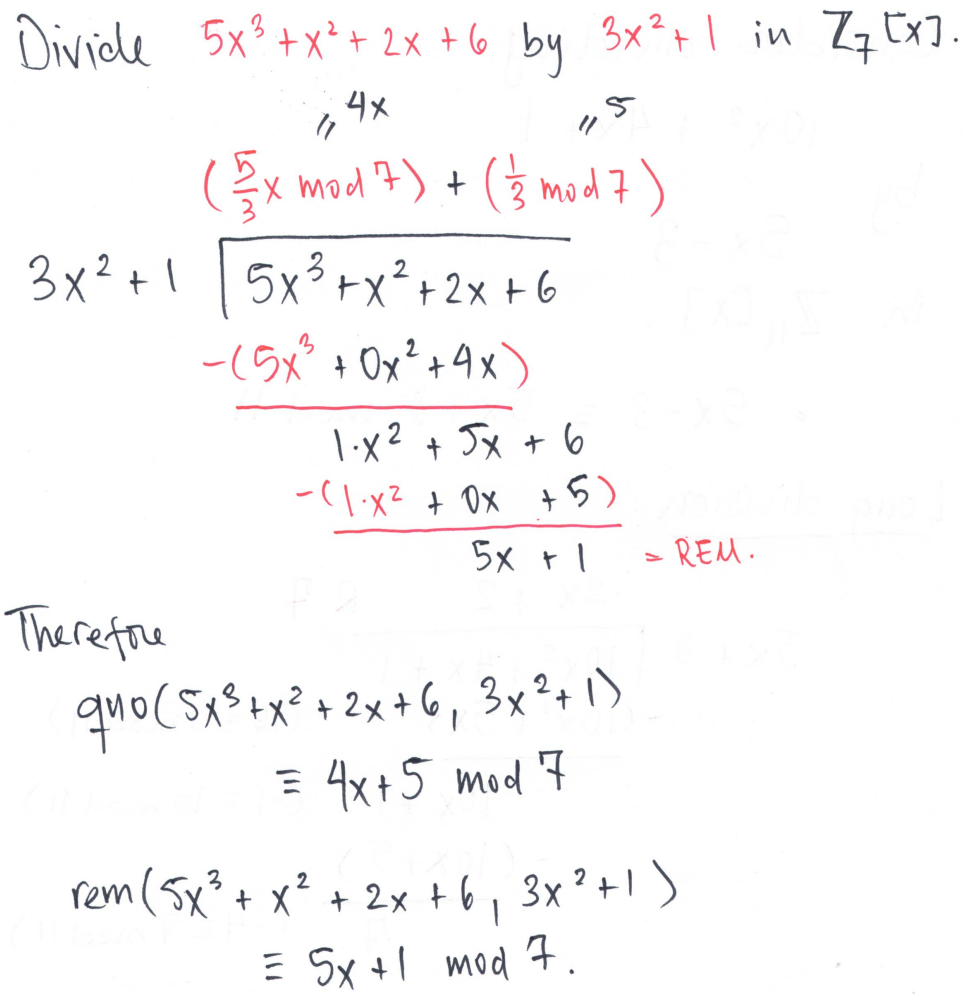
$$
\begin{array}{r|l}
 & 2x + 2 \quad R\ 7 \\
\hline
5x + 8 & 10x^2 + 4x + 1 \\
 & -(10x^2 + 5x) \qquad (16 \equiv 5 \mod 11) \\
\hline
 & 10x + 1 \qquad (-1 \equiv 10 \mod 11) \\
 & -(10x + 5) \\
\hline
 & 7 \qquad (-4 \equiv 7 \mod 11)
\end{array}
$$

THUS

$$10x^2 + 4x + 1 \equiv (5x + 8)(2x^2 + 2) + 7 \mod 11$$

quotient — remainder.

**Example 4.4** (Synthetic Division)**.**




**Example 4.5** (Synthetic Division)**.**

*Existence – The Division Algorithm.*

```
def DIVISION:
    Input   a, b in ZZ_p[x] such that b != 0
    Ouput   q, r such that a = b*q + r and deg r < deg b in ZZ_p[x]

    r, q = a, 0
    while r != 0 and deg r >= deg b do:
        t <- lt(r)/lt(b)    # division of *terms* in ZZ_p: (ax^n)/(bx^m) = a/b mod p x^(n-m), n>m
```

```
        q <- q + t
        r <- r - t*b        # deg r *must* monotonically decrease
    return (q,r)
```

□

**Example 4.6.** Consider $a = 10x^2 + 4x + 1$ divided by $b = 5x - 3$ in $\mathbb{Z}_{11}[x]$.

$$r, q \leftarrow 10x^2 + 4x + 1, 0$$

$r \neq 0$ and $\deg r = 2 \geq \deg b = 1$

$$t \leftarrow \frac{10x^2}{5x} \equiv 2x \bmod 11$$
$$q \leftarrow 0 + 2x$$
$$r \leftarrow (10x^2 + 4x + 1) - 2x(5x - 3) \equiv 10x + 1 \bmod 11$$

$r \neq 0$ and $\deg r = 1 \geq \deg b = 1$

$$t \leftarrow \frac{10x}{5x} \equiv 2 \bmod 11$$
$$q \leftarrow 2x + 2$$
$$r \leftarrow (10x + 1) - 2(5x - 3) \equiv 7 \bmod 11$$

$r \neq 0$ and $\deg r = 0 \geq \deg b = 1$

return $(2x + 2, 7)$

And notice $10x^2 + 4x + 1 \equiv (5x - 3)(2x + 2) + 7 \bmod 11$.

# 5 Heaps

Sparse polynomial arithmetic relies on the *heap* data structure to be efficient. A *heap* is a type of priority queue in which priority is determined by an ordering on the elements of the heap. The element that is *maximum* with respect to this ordering has the highest priority in the queue and can be removed most efficiently. When the maximum element has the highest priority, the heap is more precisely called a *max heap*.

**Definition 5.1** (Heap)**.** A vector of $n$ elements $H = [H_1, H_2 \ldots, H_n]$ is a *heap* when it satisfies the following properties:

$$H_i \geq H_{2i} \qquad \text{and} \qquad H_i \geq H_{2i+1} \qquad \text{for} \qquad i \geq 1$$

**Example 5.1.** If $H = [11, 7, 3, 4, 6, 5, 1]$ then $H$ is *not* a heap because $H_3 < H_6$.

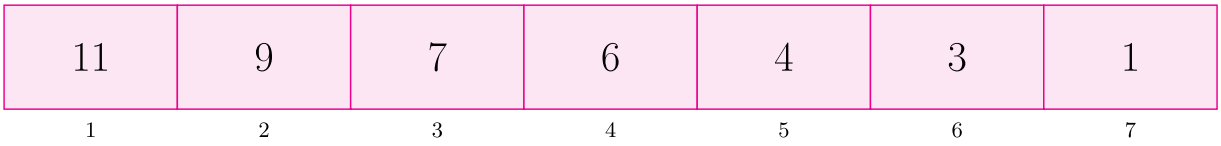
**Example 5.2.** If $H = [11, 9, 6, 7, 4, 3, 1]$ then $H$ *is* a heap.

Importantly,

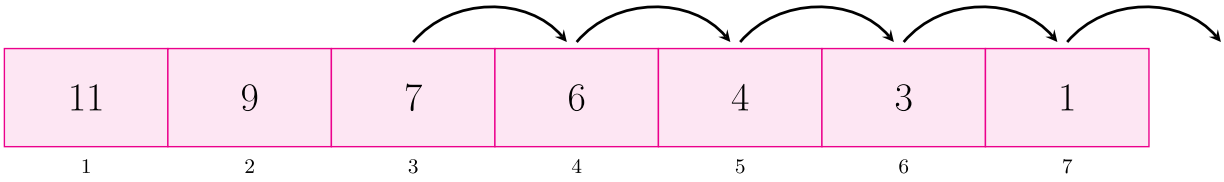
1. If $H$ is in *descenindg* order then $H$ is a heap
2. If $H$ is a heap then $H_1$ is the maximum element.

One may ask why we do not simply use a sorted list for this task. Notice, however, that inserting a new element into a sorted list may require shifting existing elements to make room. The cost of this operation is proportional to the size of the list, whereas insertion into a heap has a cost proportional to the logarithm of its size.

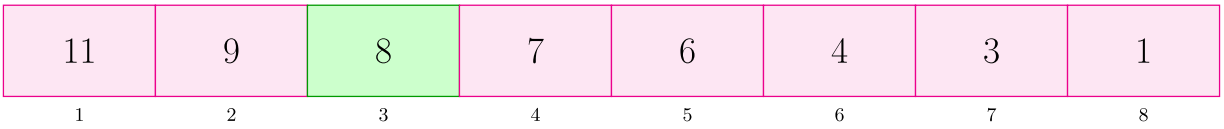
**Example 5.3.** Adding the element 8 to a queue encoded by a sorted list.




(a) Step 1: Initial queue.




(a) Step 2: Make room for the new element by shifting elements greater than 8.




(a) Step 3: Insert new element.

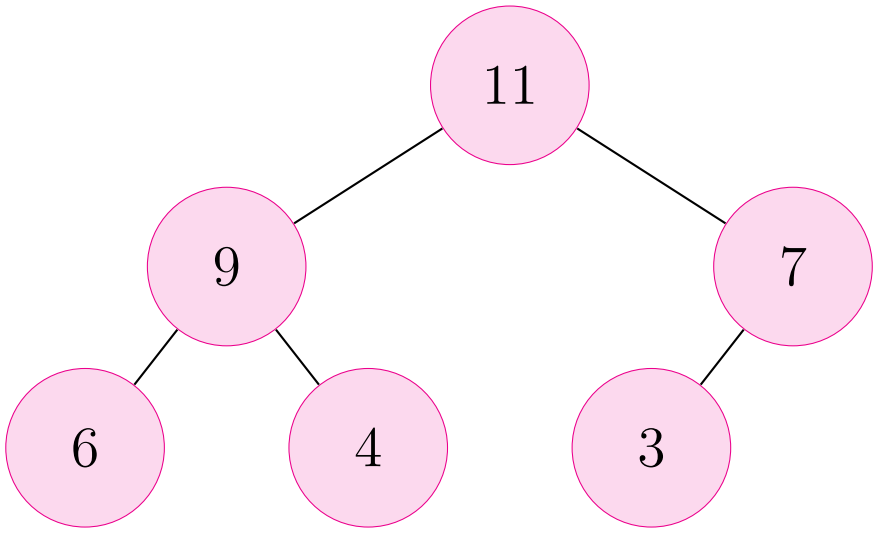
## 5.1 Underlying Representation

We visualize heaps as *binary trees*

When using this representation, the maximum element is the *root* of the tree, and the heap satisfies the recursive property that every subtree is also a heap. This means that every node in the heap is larger than (or equal to) each of its descendants.

One may ask why we do not instead use a sorted this for this task. But notice that adding (or removing) something from an ordered list will require shifting elements. A task with expense proportional to the size of the list.

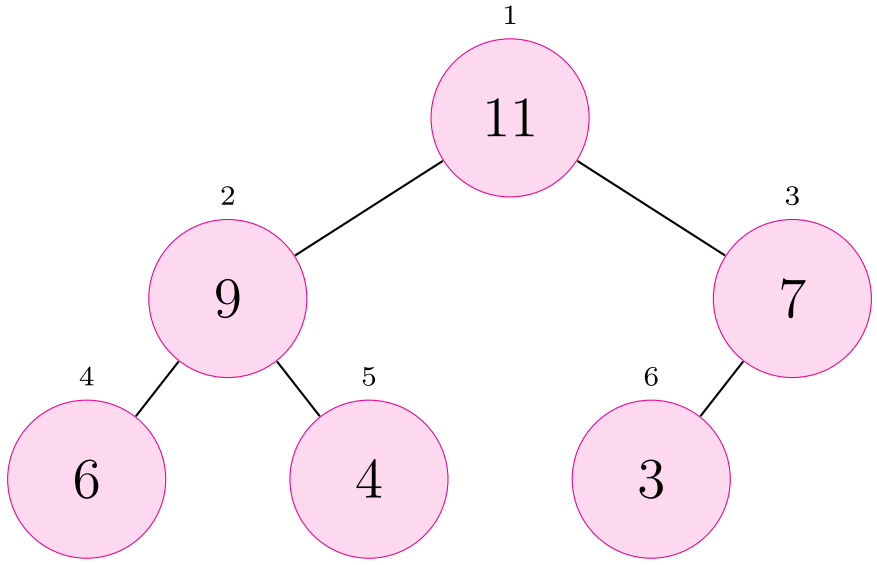
**Example 5.4.** The heap $H = [11, 9, 6, 7, 4, 3, 1]$ as a binary tree.




Notice that node 2 has children $2 \cdot 2 = 4$ and $2 \cdot 2 + 1 = 5$ and parent $\lfloor \frac{2}{2} \rfloor = 1$.

If we number the nodes of the tree from the top-to-bottom and left-to-right the node numbers correspond to exactly their position in the vector representation. This means 1. The *children* of node $i$ are at positions $2i$ and $2i + 1$, 1. The parent of node $i$ is at postion $\lfloor \frac{i}{2} \rfloor$.

**Example 5.5.** The heap $H = [11, 9, 6, 7, 4, 3, 1]$ as a binary tree.




A heap $H$ with $n = |H|$ elements supports the following operations.

Table 1: Basic heap operations and their complexities.

| Operation | Description | Heap |
|---|---|---|
| Create | Create a new heap with $n$ elements | $O(n \log n)$ |
| Peek | Query the value of the top of the heap. | $O(1)$ |

| Operation | Description | Heap |
|---|---|---|
| Push | Insert a new element into the heap. | $O(\log n)$ comparisons |
| Pop | Remove the maximum element from the heap. | $O(\log n)$ comparisons |
| Length | Return the number of elements in the heap. | $O(1)$ |
| Empty(H) | Query if there is at least one element in the heap. | $O(1)$ |

Table 2: Comparison of array and heap operations.

| Operation | Vector | Heap |
|---|---|---|
| Create | $O(n \log n)$ | $O(n \log n)$ |
| Push | $O(n)$ | $O(\log n)$ |
| Pop | $O(n)$ | $O(\log n)$ |

## 5.2 Implemeting a Heap in Julia

A `heap` will be a new struct with a `data` attribute for storing the heap's element.

```
struct Heap{T}
    data :: Vector{T}

    # Inner constructor for initializing empty heap
    function Heap{T}() where T
        new(Vector())
    end
end
```

```
h = Heap{Int64}()
dump(h)
```

```
Heap{Int64}
  data: Array{Int64}((0,)) Int64[]
```

For the time being, so we can run examples, we will let hepas be created from arbitrary vectors that we assume satisfy the heap property (though we wont enforce it).

```
struct Heap{T}
    data::Vector{T}

    # Inner constructor for initializing empty heap
    function Heap{T}() where T
        new(Vector())
    end

    # Inner constructor for initializing a heap a vector v
    function Heap(vec::Vector{T}) where T
      # precondition:  vec[i]   vec[2i], vec[2i+1] for i   1
      new{T}(vec)
    end
end
```

```
h = Heap([11, 9, 7, 6, 4, 3])
dump(h)
```

```
Heap{Int64}
  data: Array{Int64}((6,)) [11, 9, 7, 6, 4, 3]
```

### 5.2.1 Base Interfaces

There are some standard interfaces for containers that we can/should define now for `Heap` struct. Most of them are wrappers over the `Vector` that holds the heap's data.

```
Base.length(heap::Heap) = length(heap.data)
```

```
length(h)
```

```
6
```

```
Base.isempty(heap::Heap) = isempty(heap.data)
```

```
isempty(h)
```

```
false
```

```
isempty(Heap{Int64}())
```

```
true
```

The following will enable us to write `h[3] = 7` instead of `h.data[3] = 7` when `h isa Heap`.

```
Base.firstindex(heap::Heap) = heap.data |> firstindex
Base.lastindex(heap::Heap) = heap.data |> lastindex
Base.getindex(heap::Heap, k::Integer) = heap.data[k]

function Base.setindex!(heap::Heap{T}, value::T, key::Integer) where T

    key > length(heap) && error("Out of bounds")
    key < 0 && error("Out of bounds")

    heap.data[key] = value
end
```

```
firstindex(h), lastindex(h), getindex(h[begin]), getindex(h[end])
```

```
(1, 6, 11, 3)
```

```
h[end] = h[end] + 1
getindex(h[end])
```

```
4
```

```
peek(heap::Heap) = heap[begin]
```

```
peek(h)
```

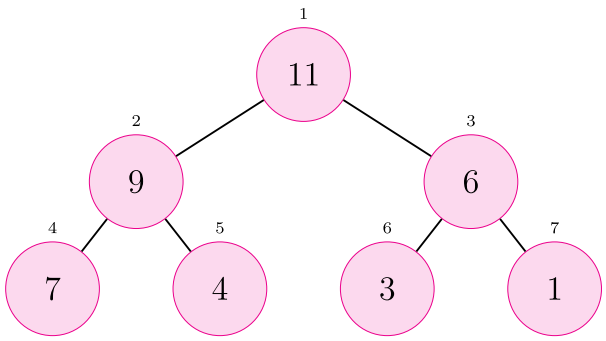
```
11
```

### 5.2.2 Pop / Sink

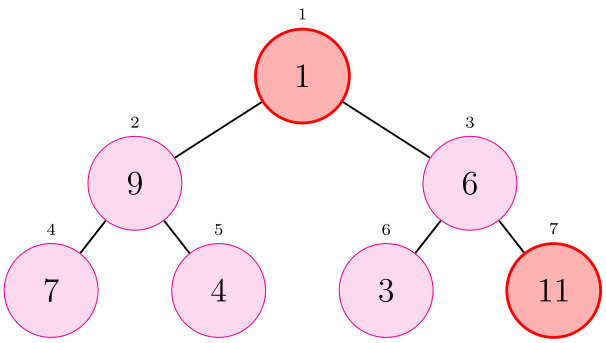
To "pop" from the heap is to remove its largest element and then restore the heap property.

To "sink" is to let an element move downward (by swapping with the largest child) until the heap property is restored. When we pop from the heap we replace its root with the last leaf and then let it sink.

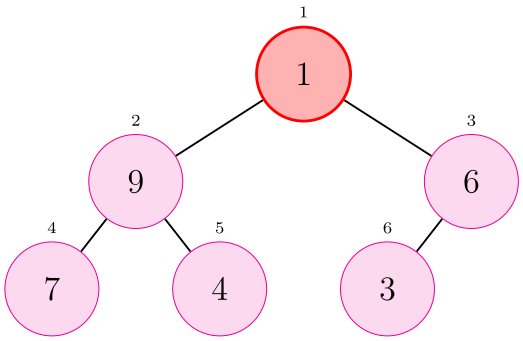
**Example 5.6.** Popping `11` then sinking `1` through the heap so that the result is a `heap`.



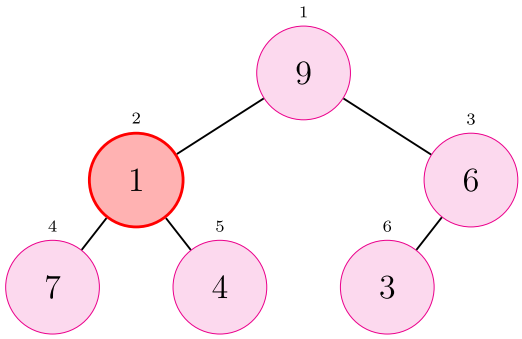

(a) Step 1: Initial heap.



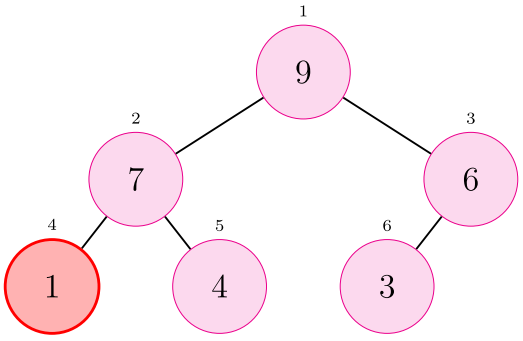

(a) Step 2: Swap the head and last leaf.




(a) Step 3: Remove/pop the last leaf.




(a) Step 4: Sink (swap with largest child).




(a) Step 5: Sink (swap with largest child).

```
"""
Sink node k in the heap.

Each sink restores the heap property for the subtree rooted at that node.
"""
function sink!(heap, k)
    n = length(heap)
    while 2k  n  # current node has (at least) left child
```

```
        child_index = 2k

        # check if right child exists and is bigger than left child
        if child_index+1  n && heap[child_index] < heap[child_index+1]
            child_index += 1
        end

        heap[k]  heap[child_index] && break  # cannot sink

        heap[k], heap[child_index] = heap[child_index], heap[k]

        k = child_index
    end

    nothing
end
```

```
function Base.pop!(heap::Heap)
  heap[begin], heap[end] = heap[end], heap[begin]  # swap root with last leaf
  head = pop!(heap.data)  # pop the leaf
  sink!(heap, 1)
  head
end
```

```
h = Heap([11, 9, 6, 7, 4, 3, 1])
while !isempty(h)
    println(pop!(h))
end
```

```
11
9
7
6
4
3
1
```

**Proposition 5.1** (Complexity)**.** *Pop does* $O(\log n)$ *many comparisions.*

*Proof.* The maximum number of swaps possible is equal to the depth of the tree. The depth of the tree with $n$ nodes is $\log_2 n$. □

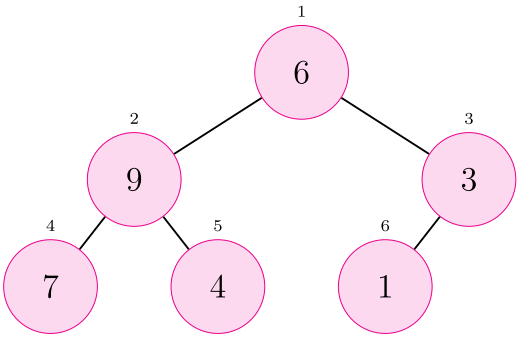
### 5.2.3 Push / Swim

To "swim" is to let a node rise in the tree as high as possible while mantaining the heap property. An element is inserted into the heap by making it the next leaf and letting it swim.

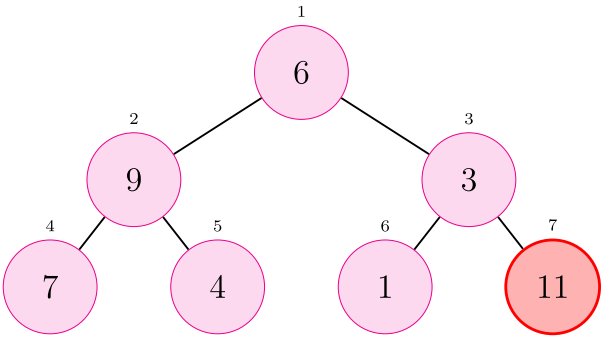
**Example 5.7.** Inserting `11` into the heap.

```
function Base.push!(heap::Heap{T}, x::T) where T
    push!(heap.data, x)
    k = length(heap)

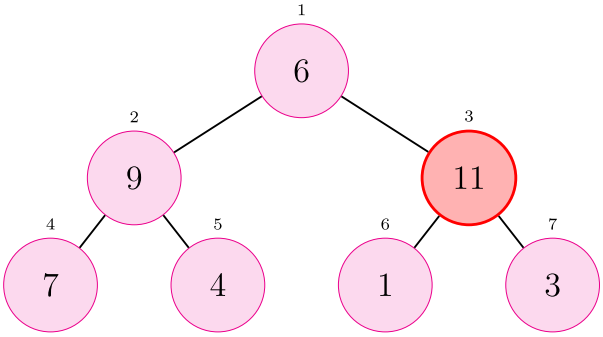
    while k > 1 # node k has a parent
        kparent = k ÷ 2
        heap[kparent] > heap[k] && return
        heap[kparent], heap[k] = heap[k], heap[kparent]
        k = kparent
        #swap
```



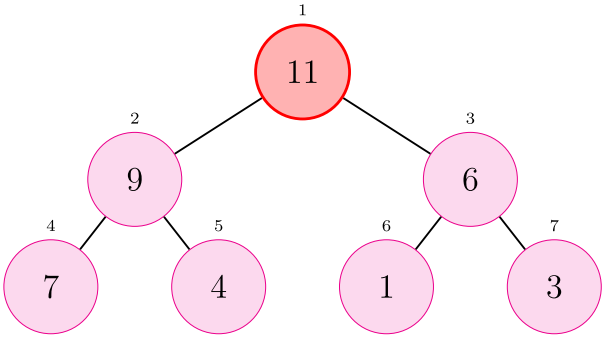

(a) Step 1: Initial heap




(a) Step 2: Insert element as last leaf.




(a) Step 3: Swim




(a) Step 4: Swim

```
    end

    nothing
end
```

```
h = Heap([6, 9, 3, 7, 4, 1])
push!(h, 11)
@show h
```

```
h = Main.Notebook.Heap{Int64}([11, 9, 6, 7, 4, 1, 3])
```

```
Heap{Int64}([11, 9, 6, 7, 4, 1, 3])
```

**Proposition 5.2** (Complexity)**.** *Insert does* $O(\log n)$ *many comparisions.*

*Proof.* The maximum number of swaps possible is equal to the depth of the tree. The depth of the tree with $n$ nodes is $\log_2 n$. □

## 5.3 Heapfiy

To *heapify* is to create a heap from an *arbitrary* vector whose elements are given in no particular order.

The strategy is to traverse the vectors backwards letting each node sink. Notice, however, that any node at index beyond the halfway point of the vector is leaf. We therefore should start traversing at the halfway point.

We revisit our inner constructor that currently assume the input input vector already satisfies the heap property and remove this precondition.

```
struct Heap{T}
    data::Vector{T}

    # Inner constructor for initializing empty heap
    function Heap{T}() where T
        new(Vector())
    end

    function Heap(vec::Vector{T}) where T
        h = new{T}(vec)
        for k in length(h)÷2:-1:1
            sink!(h, k)
        end
        h
    end
end
```

```
vec = [1, 3, 6, 4, 7, 11, 9]
Heap(vec)
```

```
Heap{Int64}([11, 7, 9, 4, 3, 6, 1])
```

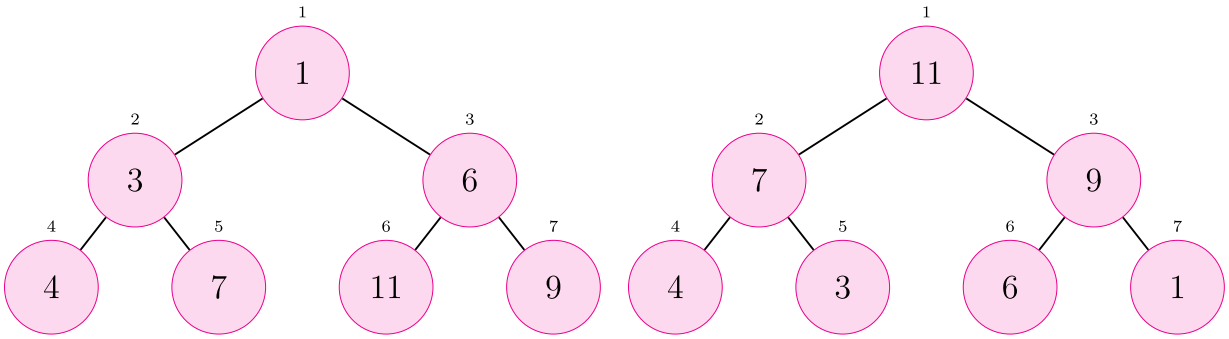



Figure 19: Before and after heap construction.